

P3662R0

Improve Increment and Decrement Operator Syntax

Published Proposal, 2025-03-14

This version:

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3662r0.html>

Author:

[Jeremy Rifkin](#)

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/jeremy-rifkin/proposals/blob/main/cpp/increment-decrement.bs>

Abstract

C++ differentiates prefix and postfix increment and decrement operator overloads with an `int` parameter or lack thereof. This syntax is subtle, not expressive, and not intuitive. This is a short paper on pursuing a more expressive syntax.

Table of Contents

1 Introduction

2 History

3 Proposal

4 Alternatives

5 Questions for EWGI

6 Proposed Wording

References

Informative References

§ 1. Introduction

C++ allows overloading the prefix and postfix `++` and `--` operators with the following syntax:

```
struct S {  
    auto operator++()      { ... } // ++s  
    auto operator++(int) { ... } // s++  
    auto operator--()      { ... } // --s  
    auto operator--(int) { ... } // s--  
};
```

This syntax is a vestige of a historical design decision that, while pragmatic at the time, isn't good language design by today's standard (and wasn't then either). It presents three main problems:

- It's subtle
- It's not expressive
- It's not intuitive
- It's not conducive to education

The current syntax was chosen for two main reasons: It was a minimal change to the language and it didn't add new keywords. Today, tools such as contextual keywords allow us to solve this problem better.

Overloading increment and decrement operators is not super common: Plenty of C++ programs are written without ever overloading these operators, with their predominant use being iterator implementations. However, it's also not an uncommon syntax to need to use. To put the scale of this topic into perspective: According to GitHub code search, `operator++` appears about [416k times](#) in open source C++ code at the time of writing. This is the same order of magnitude as `std::runtime_error`, which appears about [647k times](#) at the time of writing.

While this isn't a problem most developers run into daily, and certainly developers are able to memorize which signature is which or look up the syntax, developers should not have to use memorization tricks in order to overload these operators or remember what the syntax means when reading code. Today, we can do better. This paper proposes a modern syntax for these operators, which significantly improves the expressiveness, intuitiveness, readability, and teachability of code using these overloads and largely corrects a historical wart of the language.

§ 2. History

Originally, C++ only had `operator++()` and `operator--()`, which were used for both prefix and postfix variations. Postfix-specific overloads were added later. According to [*The Design and Evolution of C++*](#) 11.5.3, a driving motivator for the selected syntax was to find a "minimal change to express the prefix/postfix distinction." To further quote Stroustrup:

I considered the obvious solution, adding the keywords `prefix` and `postfix` to C++:

```
class Ptr_to_X {
    // ...
    X operator prefix++;    // prefix ++
    X& operator postfix++;  // postfix ++
};
```

or

```
class Ptr_to_X {
    // ...
    X prefix operator++;    // prefix ++
    X& postfix operator++;  // postfix ++
};
```

However, I received the usual howl of outrage from people who dislike new keywords. Several alternatives that did not involve new keywords were suggested. For example:

```
class Ptr_to_X {
    // ...
    X ++operator();    // prefix ++
    X& operator++;     // postfix ++
};
```

or

```
class Ptr_to_X {
    // ...
    X& operator++;     // postfix because it
                        // returns a reference
    X operator++;     // prefix because it
                        // doesn't return a reference
};
```

I considered the former too cute and the latter too subtle. Finally, I settled on:

```
class Ptr_to_X {
    // ...
    X operator++;      // prefix: no argument
```

```
X& operator++(int); // postfix: because of
                    // the argument
};
```

This may be both too cute and too subtle, but it works, requires no new syntax, and has a logic to the madness. Other unary operators are prefix and take no arguments when defined as member functions. The "odd" and unused dummy `int` argument is used to indicate the odd postfix operators. ...

The syntax, while pragmatic at the time, and while it does get the job done, is certainly subtle and unintuitive. Even at the time of writing [\[DNE\]](#), Stroustrup described it as "a bit of a wart."

Today, this syntax is a very notable wart, especially in light of syntax improvements and pushes for more modern syntax in the language. Today, a much different design decision would likely have been made, especially given tools like contextual keywords, which didn't exist at the time.

§ 3. Proposal

This paper proposes the following syntax with contextual keywords:

```
struct S {
    auto operator++ prefix() { ... } // ++s
    auto operator++ postfix() { ... } // s++
};
```

This greatly improves the clarity of code using these operator overloads.

Due to ABI, these would not be new signatures, but rather, they would be syntax sugar for the existing overload signatures:

New syntax

Same as

`operator++ prefix()`

`operator++()`

`operator++ postfix()`

`operator++(int)`

In addition to usage as `++s` and `s++`, it could be used as follows:

```
S s;  
s.operator++ prefix();  
s.operator++ postfix();  
// expands to:  
// s.operator++();  
// s.operator++(0);
```

One shortcoming of this approach, however, is that the underlying signature would still manifest when working with member function pointers:

```
auto(S::* pre)() = &S::operator++;  
auto(S::* post)(int) = &S::operator++;
```

Passing an extra `int` parameter would also be required when using the postfix member function pointer.

This could be partially alleviated with the following semantics, at the cost of some added complexity:

```
auto pre = &S::operator++ prefix;  
auto post = &S::operator++ postfix;  
// works as if by:  
// auto(S::* tmp1)() = &S::operator++;  
// auto pre = tmp1;  
// auto(S::* tmp2)(int) = &S::operator++;  
// auto pre = tmp2;
```

While it is unfortunate that the signature wart could still manifest, making member function pointers to increment/decrement operator overloads is likely rare.

§ 4. Alternatives

In a discussion on the `#include<c++>` discord, Janet Cobb suggested the following utility and workaround:

```
namespace std {
    using prefix = void;
    using postfix = int;
}

struct S {
    auto operator++(std::prefix) { ... } // ++s
    auto operator++(std::postfix) { ... } // s++
};
```

While this is only a facade over the problem (not entirely dissimilar to the proposed change, which is merely syntactic sugar), it would be a very low-complexity library change that would greatly improve the status quo without adding language complexity. In addition to usage as `++S` and `S++`, it could be used as follows:

```
S s;
s.operator++();
s.operator++(std::postfix());
auto(S::* pre)(std::prefix) = &S::operator++;
auto(S::* post)(std::postfix) = &S::operator++;
```

Unfortunately, `s.operator++(std::prefix());` would be an invalid use of a void expression, and similar to the proposed solution, an extra `int` parameter would have to be passed when using the postfix member function pointer.

§ 5. Questions for EWGI

1. Would this be better solved as a core change or a library change?

§ 6. Proposed Wording

Wording pending guidance and interest.

§ References

§ Informative References

[DNE]

Bjarne Stroustrup. *The Design and Evolution of C++*. URL:
<https://www.stroustrup.com/dne.html>